RELAZIONE

SMART STUDY AI

Fabio Cerioli

SOMMARIO

1. Introduzione e Scopo dell'Applicazione
2. Requisiti di Sistema
3. Architettura Software (MVVM)
4. Tecnologie Utilizzate e Implementazione
5. Guida all'Uso
6. Conclusioni e Sviluppi Futuri

INTRODUZIONE E SCOPO DELL'APPLICAZIONE

SmartStudyAI è un'applicazione Android pensata per assistere gli studenti nella pianificazione, gestione e monitoraggio delle proprie sessioni di studio.

Sfruttando i sensori hardware del dispositivo (come Accelerometro e Giroscopio), l'applicazione è in grado di raccogliere dati sui movimenti del telefono.
Questi dati vengono inviati a un Web Service esterno che, tramite un modello predittivo, restituisce se l'utente sta effettivamente studiando o se si sta distraendo indicando anche un punteggio di confidenza della predizione (es. utilizzando il telefono per altri scopi).

L'app permette inoltre di pianificare un calendario di studio settimanale e fornisce statistiche dettagliate sulle performance dell'utente.

REQUISITI DI SISTEMA

Per garantire il corretto funzionamento dell'applicazione, sono richiesti i seguenti requisiti:

- Sistema Operativo: Android 8.0 o superiore.
- Permessi: Sono richiesti permessi per accesso ad Internet, invio di notifiche e rilevazioni dei sensori
- Hardware: Il dispositivo deve essere dotato di sensori fisici (Accelerometro e Giroscopio).
- Rete: Connessione internet attiva per permettere la comunicazione con il Web Service di Machine Learning.

ARCHITETTURA SOFTWARE

Il progetto è stato creato utilizzando il pattern MVVM (Model-View-ViewModel).
Questo ha permesso di mantenere una forte separazione dei compiti:

- Model: Costituito dalle classi Entity (Database), dai modelli JSON per la rete e dal pacchetto sensors. I dati vengono gestiti tramite i Repository che consentono l'interazione con i dati.
- ViewModel: Funge da ponte tra l'interfaccia e i dati. Mantengono lo stato della UI in modo reattivo utilizzando le StateFlow e le Kotlin Coroutines, sopravvivendo ai cambi di configurazione.
- View: L'interfaccia utente è interamente dichiarativa, sviluppata utilizzando Jetpack Compose.

TECNOLOGIE UTILIZZATE E IMPLEMENTAZIONE

- Interfaccia Grafica e Navigazione (Jetpack Compose)
- Database Locale (Room)
- Comunicazione di Rete (Retrofit)
- Hardware e Machine Learning (SensorManager)
- Gestione Notifiche (AlarmManager)

INTERFACCIA GRAFICA E NAVIGAZIONE

L'intera UI è stata realizzata con Jetpack Compose.
L'interfaccia reagisce istantaneamente ai cambiamenti di stato grazie all'uso di collectAsState(),
permettendo ad esempio di visualizzare il timer della sessione che si aggiorna in tempo reale.
La navigazione tra le schermate è gestita da Navigation Compose (NavHost). È stata creata una classe Screen per definire le rotte.

```
sealed class Screen(
    val route: String
) {

    object Home : Screen("home")
    object Create : Screen("create")
    object Edit : Screen("edit")
    object Session : Screen("session")
    object Calendar : Screen("calendar")
    object Statistics : Screen("statistics")

}
```

DATABASE LOCALE

La gestione dei dati è affidata alla libreria Room di Android Jetpack.

Il database sottostante è strutturato in tre entità principali: StudyActivity, StudySchedule e StudySession.

È stata implementata l'integrità relazionale: le tabelle Schedule e Session sono collegate ad Activity tramite l'annotazione @ForeignKey con opzione onDelete = CASCADE.

```
@Entity(
    tableName = "study_sessions",
    foreignKeys = [
        ForeignKey(
            entity = StudyActivity::class,
            parentColumns = ["activityId"],
            childColumns = ["activityId"],
            onDelete = ForeignKey.CASCADE
        )
    ]
)
```

COMUNICAZIONE DI RETE

Per dialogare con il server, è stata integrata la libreria Retrofit accoppiata a Gson.

```
data class WindowData(
    val samples: List<SensorData>
)
```

Tramite chiamate asincrone, l'app invia un payload WindowData e riceve una PredictionResponse con la predizione generata dal modello e la relativa confidenza percentuale.

```
data class PredictionResponse(

    val prediction: String,
    val confidence: Double,
    val features: Int
)
```

HARDWARE E MACHINE LEARNING

- Tramite il SensorManager di Android, il SensorCollector si mette in ascolto degli eventi di TYPE_ACCELEROMETER e TYPE_GYROSCOPE con un intervallo di 20 millisecondi (50 Hz).
- I dati vengono immagazzinati nella classe WindowBuffer, la quale raggruppa i campioni in array di 128 elementi (finestra temporale) e, solo a riempimento completato, li espone al Repository per l'inoltro al server, ottimizzando le chiamate di rete.

```
data class SensorData(
    val timestamp: Long,

    val ax: Float,
    val ay: Float,
    val az: Float,

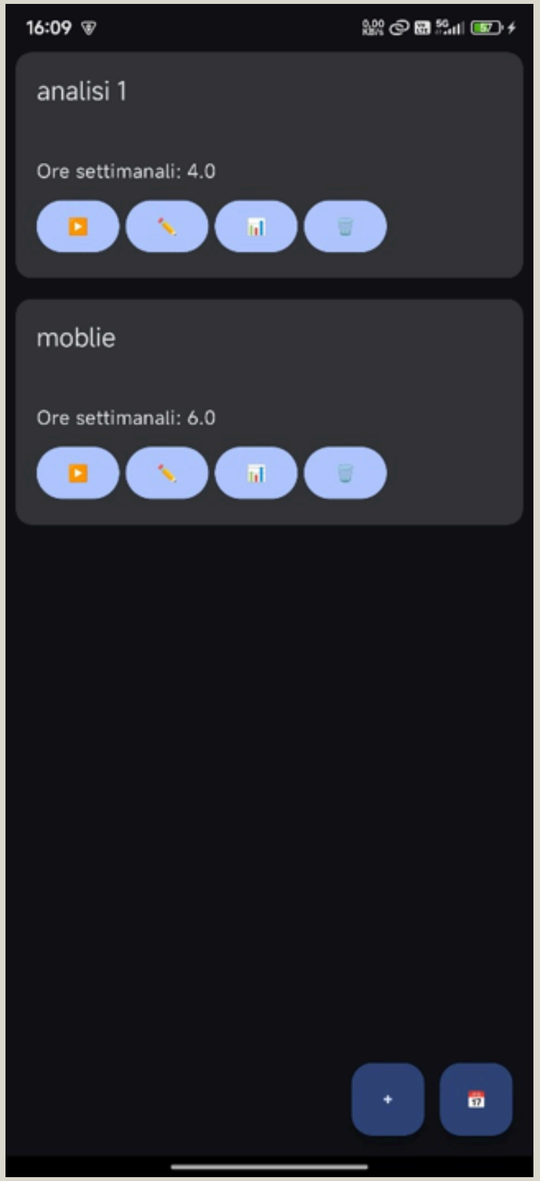
    val gx: Float,
    val gy: Float,
    val gz: Float
)
```

GESTIONE NOTIFICHE

L'applicazione avvisa l'utente quando è il momento di studiare.
Per farlo, si avvale dell'AlarmManager e di un BroadcastReceiver.
Gli allarmi sono impostati per ripetersi settimanalmente.

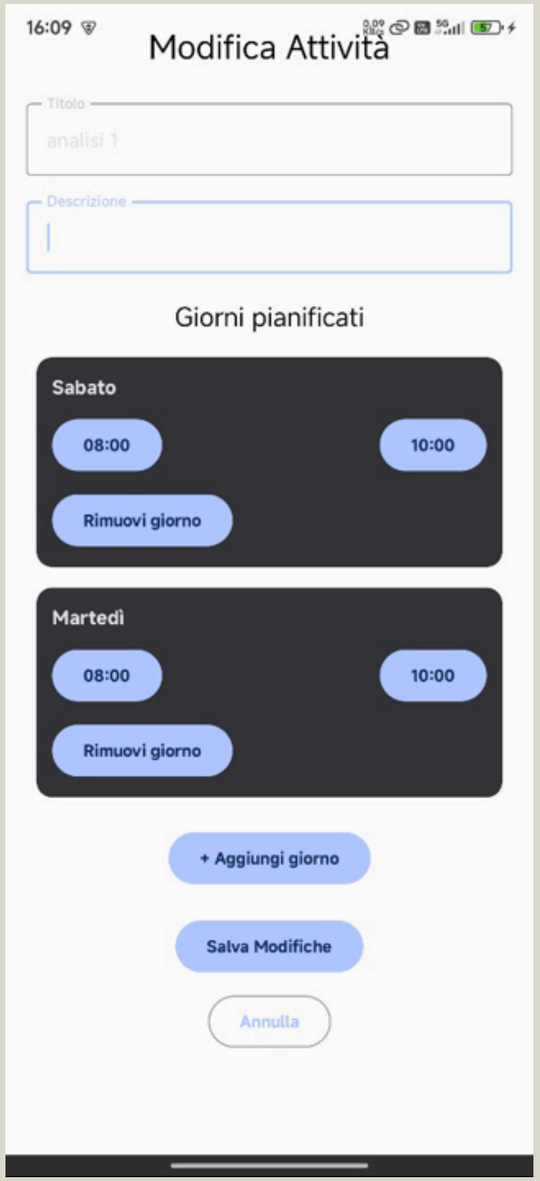
GUIDA ALL'USO: HOME

All'apertura, l'utente visualizza le attività di studio salvate.
Da qui può avviare una sessione, visualizzare le statistiche, modificare o eliminare un'attività.



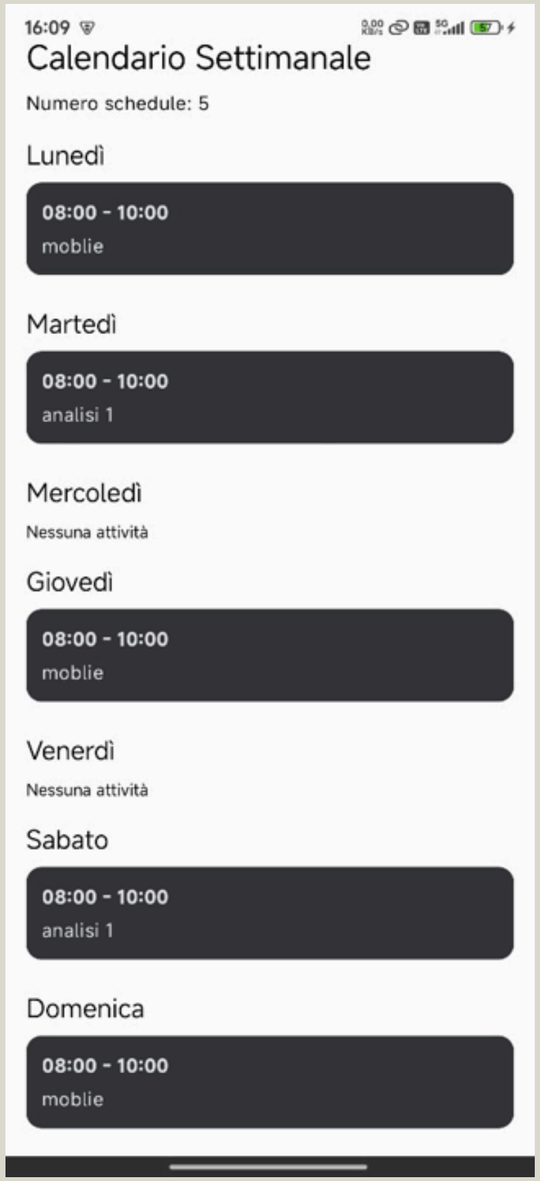
GUIDA ALL'USO: ATTIVITÀ

Creazione/Modifica:
L'utente inserisce titolo, descrizione e definisce l'orario di studio.
Il sistema calcola in automatico il monte ore settimanale



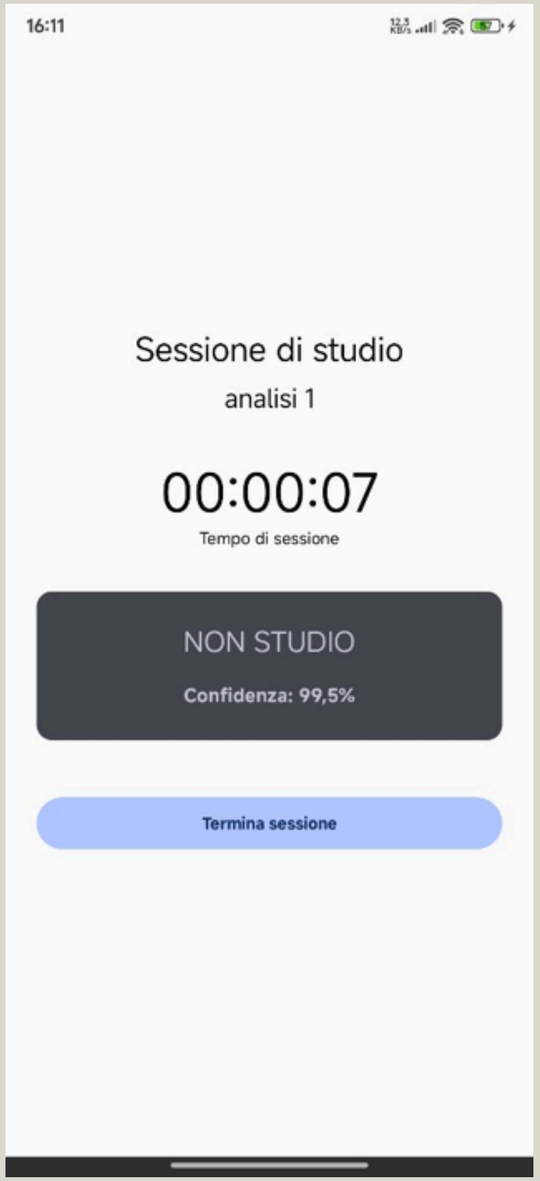
GUIDA ALL'USO: CALENDARIO

Una schermata riassuntiva raggruppa e mostra gli impegni schedulati per i 7 giorni della settimana



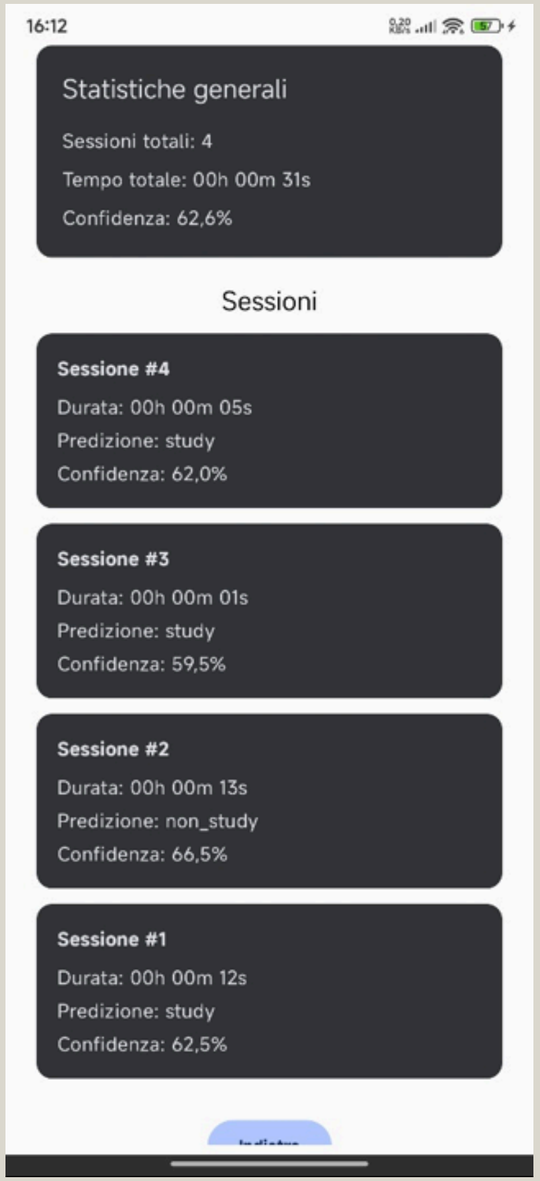
GUIDA ALL'USO: STUDIO

Una volta avviata la sessione, parte un cronometro. In background, i sensori iniziano a campionare i dati e l'interfaccia mostra in tempo reale se l'IA considera l'utente "In Studio" o "Distratto", aggiornando il valore percentuale di confidenza.



GUIDA ALL'USO: STATISTICHE

Permette di revisionare l'attività storica, visualizzando il numero di sessioni totali, il tempo speso e la qualità dello studio (media della confidenza registrata dall'IA).



GUIDA ALL USO: CONFIGURAZIONE

Prima dell'avvio della applicazione è necessario inserire l'indirizzo ip privato fornito dal terminale all'avvio del server all itnerno della pagina ApiClient.

```
PS C:\Users\cerio\Desktop\smartstudyai\Backend> py .\app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8000
 * Running on http://192.168.0.2:8000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 720-918-389
```

```
object ApiClient {


    private const val BASE_URL ="http://192.168.0.2:8000/"
```

CONCLUSIONI E SVILUPPI FUTURI

L'obiettivo iniziale di creare un ecosistema di studio potenziato dall'IA è stato raggiunto. L'app risulta funzionale, fluida e dimostra la fattibilità di unire l'IoT dello smartphone (sensori) a servizi di backend remoti per l'analisi comportamentale.
Sviluppi futuri:

- Miglioramento modello ML
- Pagina statistiche piu dettagliata

FINE